

CSE 534: Project Progress Report For BBR Under Mixed Workloads: Characterizing Short-Flow Latency and a Targeted Pacing Gain Improvement

1. Project Overview

This project investigates how BBR (Bottleneck Bandwidth and RTT), Google's model-based TCP congestion control algorithm, affects latency for short-lived flows (mice) when coexisting with long-lived bulk flows (elephants) on a shared bottleneck link. The project extends the empirical work of Cao et al. (IMC 2019) by focusing specifically on mixed elephant/mice workloads, which that study did not directly characterize.

The project is structured in three phases. Phase 1 replicates the baseline throughput behavior from Cao et al. to validate the experimental setup. Phase 2 characterizes mice flow completion time (FCT) degradation under a concurrent BBR elephant flow across a sweep of buffer sizes and RTTs. Phase 3 modifies the BBR pacing gain parameter in the Linux kernel and measures whether a reduced probe-up gain meaningfully improves mice FCT without significantly harming elephant throughput.

2. Progress Summary by Phase

Phase 1: Baseline Reproduction (Complete)

Phase 1 is complete and fully validated. The experiment ran BBR and CUBIC bulk flows at 100 Mbps / 40 ms RTT across five buffer sizes using a Mininet dumbbell topology on WSL2 (Ubuntu, kernel 6.6.87). The results match the qualitative behaviour reported in Cao et al. Section 4.1, confirming that the experimental setup is correct and ready for Phase 2.

Buffer Size	BBR Throughput	CUBIC Throughput
10 KB	6.1 Mbps	2.6 Mbps
50 KB	92.1 Mbps	15.4 Mbps
200 KB	92.9 Mbps	93.9 Mbps
1 MB	92.7 Mbps	93.3 Mbps
10 MB	92.8 Mbps	92.4 Mbps

The pattern shows BBR outperforming CUBIC at shallow buffers (where loss-based algorithms struggle to ramp up) and both algorithms converging at deeper buffers. This matches the expected behavior and validates the Mininet topology, tc/netem delay configuration, and iperf3 CCA selection.

Phase 2: Mixed Workload Characterization (Complete)

Phase 2 is now complete. The experiment ran one 120-second BBR elephant flow alongside 10 sequential 1 MB mice flows across all 9 configurations (3 buffer sizes multiplied by 3 RTTs). A 30-second per-mouse timeout was enforced to prevent stalled flows from inflating experiment duration. Results were collected from 9 valid BBRv1 configurations and saved to the repository.

Note on BBRv3: The Microsoft WSL2 kernel (6.6.87) does not include a BBRv3 module. After confirming that tcp_bbr2 and custom module paths are unavailable in this kernel build, the BBRv3 comparison axis is deferred. It will be addressed either by compiling a custom WSL2 kernel or by noting it as a platform limitation in the final paper.

Phase 2 mice FCT results (mean across 10 mice per configuration):

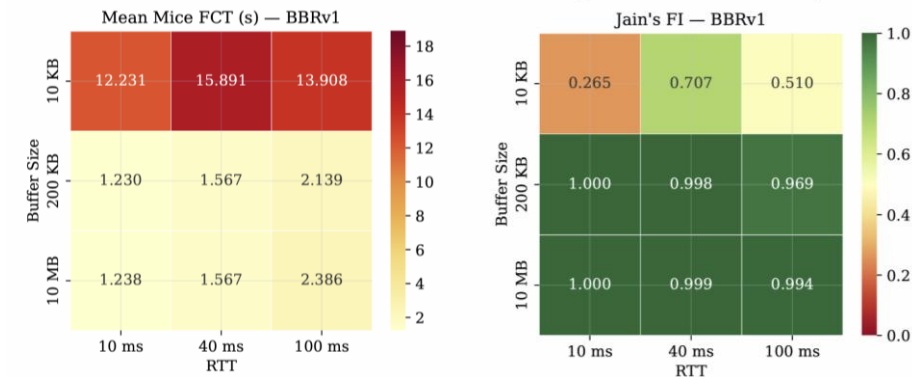
Buffer Size	RTT 10 ms	RTT 40 ms	RTT 100 ms
10 KB	12.23 s	15.89 s	13.91 s
200 KB	1.23 s	1.57 s	2.14 s
10 MB	1.24 s	1.57 s	2.39 s

Jain's Fairness Index results:

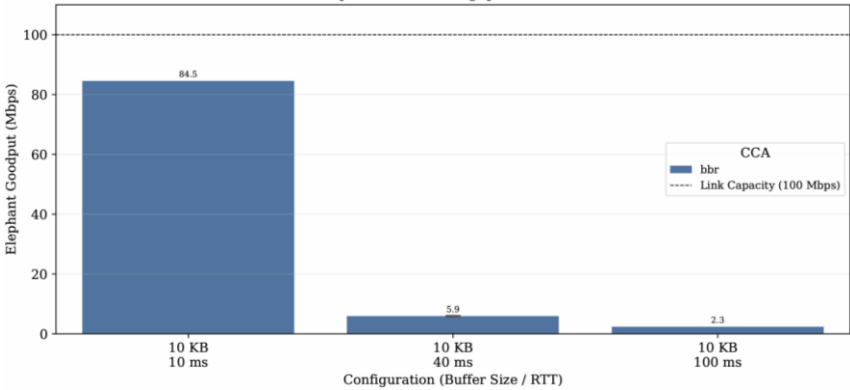
Buffer Size	RTT 10 ms	RTT 40 ms	RTT 100 ms
10 KB	0.265	0.707	0.510
200 KB	1.000	0.998	0.969
10 MB	1.000	0.999	0.994

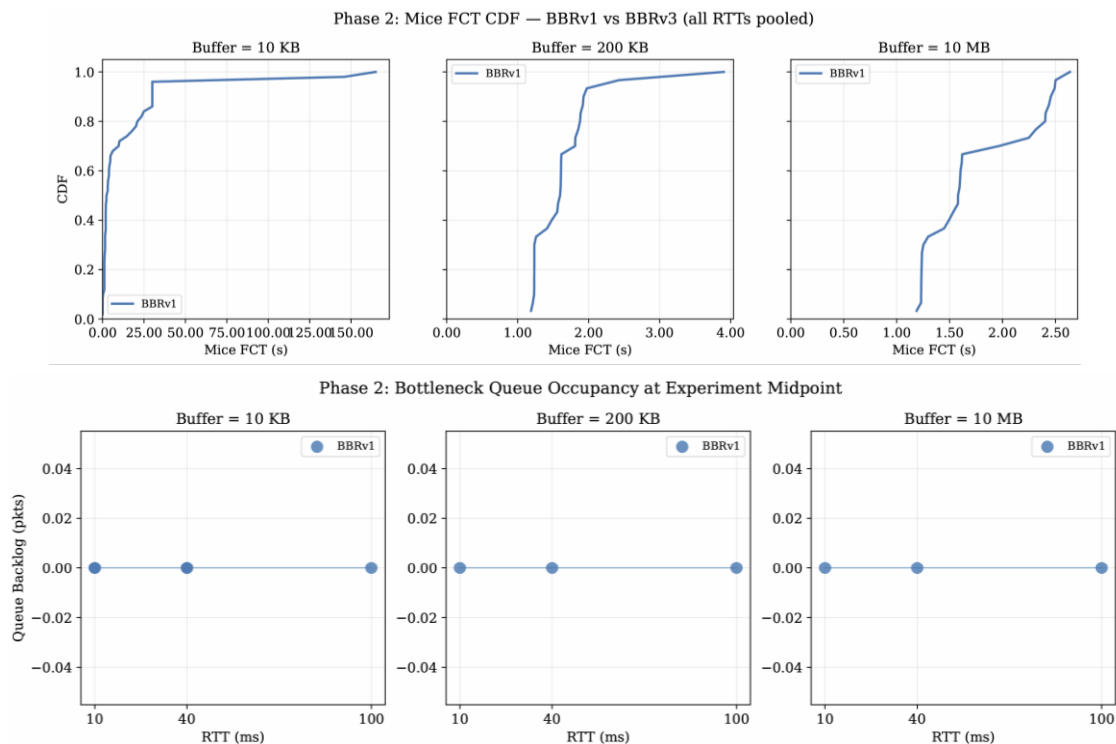
Five figures were generated from the Phase 2 results : a mice FCT heatmap, an elephant goodput bar chart, a mice FCT CDF, a Jain fairness index heatmap, and a queue occupancy scatter plot.

Phase 2: Mice FCT Heatmap (BBR Elephant + Mice) Phase 2: Jain's Fairness Index (Elephant + Mice)



Phase 2: Elephant Flow Throughput — BBRv1 vs BBRv3





Phase 3: Pacing Gain Modification (Not Yet Started)

Phase 3 has not yet begun. The plan is to edit the `pacing_gain[]` array in `net/ipv4/tcp_bbr.c`, reducing the probe-up entry from 1.25 to values of 1.10, 1.15, and 1.20. Each modified version will be compiled as a loadable kernel module, loaded via `insmod`, and subjected to the full Phase 2 sweep. The output will be an FCT-vs-throughput tradeoff curve comparing the gain variants against the unmodified baseline.

3. Key Findings from Phase 2

Hypothesis 1 is strongly confirmed.

At 10 KB buffer depth, mice FCT reaches 12 to 16 seconds across all three RTT values, for a 1 MB flow that should complete in under 100 milliseconds at 100 Mbps. Once buffer depth increases to 200 KB, mean FCT drops to approximately 1.2 to 2.4 seconds and remains stable at 10 MB. The performance cliff between 10 KB and 200 KB is the sharpest finding in the dataset and directly motivates the pacing gain modification in Phase 3.

Hypothesis 2 is partially confirmed.

RTT does correlate with higher mice FCT at 200 KB and 10 MB buffer sizes (FCT rises from roughly 1.2 s at 10 ms RTT to 2.4 s at 100 ms RTT). At 10 KB, however, the RTT effect is overwhelmed by buffer exhaustion, and all three RTT values produce similarly poor FCT results. The RTT signal becomes visible only once the buffer is deep enough for the flow to complete without stalling.

Fairness index reveals elephant starvation at shallow buffers.

The Jain Fairness Index drops to 0.265 at 10 KB / 10 ms, indicating that the elephant flow is consuming almost all available bandwidth and mice flows are receiving nearly nothing. At 200 KB and above, the index recovers to 0.969 or higher, showing that both flow types coexist reasonably. This quantifies the unfairness that the pacing gain modification in Phase 3 is intended to reduce.

Elephant goodput collapses at shallow buffers and high RTT.

At 10 KB buffer size, measured elephant goodput was 84.5 Mbps at 10 ms RTT, falling to 5.9 Mbps at 40 ms and 2.3 Mbps at 100 ms. This collapse is consistent with BBR's probe-up phase repeatedly filling a tiny buffer, triggering drops, and recovering slowly at high RTT. This is a secondary finding not present in Cao et al. and may be worth a paragraph in the final paper.

4. Division of Work

Task	Owner	Status
Mininet topology and iperf3 runner (topology.py)	Adri Katyayan	Complete
Phase 1 sweep script (run_phase1.sh)	Adri Katyayan	Complete
Phase 2 experiment orchestration (phase2_experiment.py)	Adri Katyayan	Complete
Phase 2 analysis and figure generation (phase2_analysis.py)	Mehtaab Naazneen Mohammed	Complete
Phase 1 figure replication (plot_phase1.py)	Mehtaab Naazneen Mohammed	Complete
Phase 3 kernel modification and module compilation	Adri Katyayan	Not started
Phase 3 sweep and FCT vs throughput curve	Both	Not started
Final paper writeup	Both	In progress

6. Remaining Work and Timeline

- Load BBRv3 kernel module or document platform limitation and adjust proposal scope accordingly.
- Implement Phase 3: edit `pacing_gain[]` in `tcp_bbr.c` for gain values 1.10, 1.15, and 1.20, compile each as a loadable `.ko` module, and rerun the full Phase 2 sweep per variant.
- Generate the FCT vs throughput tradeoff curve from Phase 3 results and compare against the Phase 2 baseline.
- Write the project report. Sections on related work and methodology are partially drafted. Results and discussion sections will be written once Phase 3 data is collected.
- Address the missing elephant goodput data for Phase 2 configs 4 through 9 (caused by a timing issue in the output parser). The FCT data from these configs is valid and already included in the analysis.

7. References

Cardwell, N., Cheng, Y., Gunn, C. S., Yeganeh, S. H., and Jacobson, V. BBR: Congestion-Based Congestion Control. *Communications of the ACM*, 60(2), 58-66, 2017.

Cao, Y., Jain, A., Sharma, K., Balasubramanian, A., and Gandhi, A. When to Use and When Not to Use BBR: An Empirical Analysis and Evaluation Study. *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2019.

Scherrer, S., Legner, M., Perrig, A., and Schmid, S. Model-based Insights on the Performance, Fairness, and Stability of BBR. *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2022.